

ME2400 PROJECT

#B-111

"Mini Segway"

#B-111 Members

ME20B122 - Nishant Sahoo

ME20B095 - M Keerthy Vasani

ME20B093 - Kalla Suresh

Synopsis

- | | |
|--|--|
| i) <i>Introduction</i> | x) <i>Controller Design</i> |
| ii) <i>Objective</i> | xi) <i>PID Controller</i> |
| iii) <i>Components</i> | xii) <i>Simulink Model for PID</i> |
| iv) <i>Bill</i> | xiii) <i>Controller</i> |
| v) <i>Expenditure</i> | xiii) <i>Controller Tuning</i> |
| vi) <i>Dimensions</i> | xiv) <i>Control System Block Diagram</i> |
| vii) <i>Electronic Circuit Diagram</i> | xv) <i>Arduino Program</i> |
| viii) <i>Explanation of Electronic Circuit Diagram</i> | xvi) <i>Conclusion</i> |
| ix) <i>Mechanical Design of System</i> | xvii) <i>Team Members' Contribution</i> |
| | xviii) <i>References and Software Used</i> |

Introduction

“Mini Segway” as the name suggests is a robot that balances on two wheels placed horizontally about an axis. The main design principle used in designing and constructing this robot is that of “inverted pendulum” where the concentration of the mass is at the top so that it is not easily swayed. The components are placed in such a way to minimize effects due to large disturbances.

All the relevant information can be found at our [Google Drive](#).

Objective

The objective of this project is to design and construct a Segway type robot that balances itself on two horizontally placed wheels without large swaying and the base not touching the ground. This way the robot "self-balances", in essence it can balance anything that is placed on top with reasonable weight with ease.

Components

- 1) *Arduino Uno R3*
- 2) *2 x N20-12V-300 rpm Metal Gear Motor*
- 3) *Jumper Wires (Male to Male and Male to Female)*
- 4) *USB Cable to connect to Arduino*
- 5) *L298N Driver module*
- 6) *MPU-6050 Combined Accelerometer and Gyroscope Sensor module*
- 7) *12 x 1.5V AA batteries*
- 8) *2 x Reference chassis*

- 9) 2 x Mounting Brackets to hold the geared motor
- 10) 2 x battery cases
- 11) 2 x 44mm diameter tyres

All the components used in this project are purchased from [robustools.com/](https://www.robustools.com/) website.

Disclaimer : Prices indicated (in the next page) are correct at the time of purchase. Some components are bought extra to ensure availability of components to work with incase of component failure.

Bill



All Categories



₹ 0.00

Robu Wallet

Order Invoices

Robu Point

Your Waitlists

Customer Logout

Product

Total

Arduino Uno R3 with Cable × 1

₹ 829.00

L298N 2A Based Motor Driver Module – Good Quality × 2

₹ 258.00

MPU-6050 3-Axis Accelerometer and Gyro Sensor × 3

₹ 567.00

65pcs Flexible Breadboard Jumper Wires × 1

₹ 103.00

Transparent 830 Points Solderless Breadboard × 1

₹ 149.00

N20-12V-300 Rpm Micro Metal Gear Motor × 3

₹ 807.00

Mounting Bracket for N20 Micro Gear motors-2pcs × 2

₹ 130.00

3PI miniQ Car wheel Tyre 44mm N20 DC Gear Motor Wheel × 3

₹ 225.00

Transparent Robot Smart Car- Chassis Only × 3

₹ 333.00

M5 X 10mm Male-Female Brass Hex Threaded Pillar Standoff Spacer- 12 Pcs. × 2

₹ 490.00

EasyMech CHHD M3 X 8mm Bolt and Nut Set-25 pcs. × 1

₹ 59.00

6 × 1.5V AA Battery Holder Without Cover × 2

₹ 98.00

9V 10cm Battery Connector - 5pcs × 1

₹ 39.00

Subtotal:

₹ 4,087.00

Shipping:

STANDARD SHIPPING

Delivery will take as per std shipping policy upto 12 days

Cash Handling Charges:

₹ 25.00

Payment method:

Cash on Delivery



Expenditure

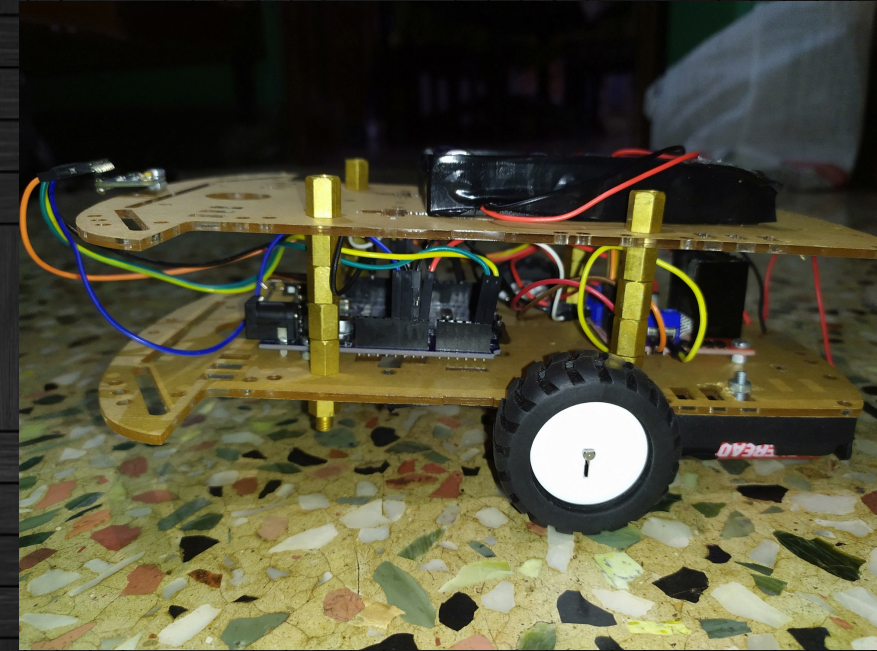
S.No	Product	Quantity	Unit Price (MRP)	Total Price (MRP)
1.	Arduino Uno R3	1	829	829
2.	L298N Motor Driver Module	1	129	129
3.	MPU-6050 Sensor	1	189	189
4.	Connecting Wires (65 pieces)	1	103	103
5.	N20 12V Geared Motor	2	269	538
6.	Mounting bracket for N20	2	65	130
7.	44mm Diameter rubber tyres	2	75	150
8.	Reference Chassis	2	111	222
9.	Nuts and Bolts (25 pieces)	1	59	59
10.	6 x 1.5 V AA battery case	2	49	98
11.	1.5V AA battery	12	9.50	114
After Using COUPON Code ROBU5 to get 5% discount => Total = 2433 Rupees				

The total expense for this project did not exceed 2500 Rupees.

Dimensions

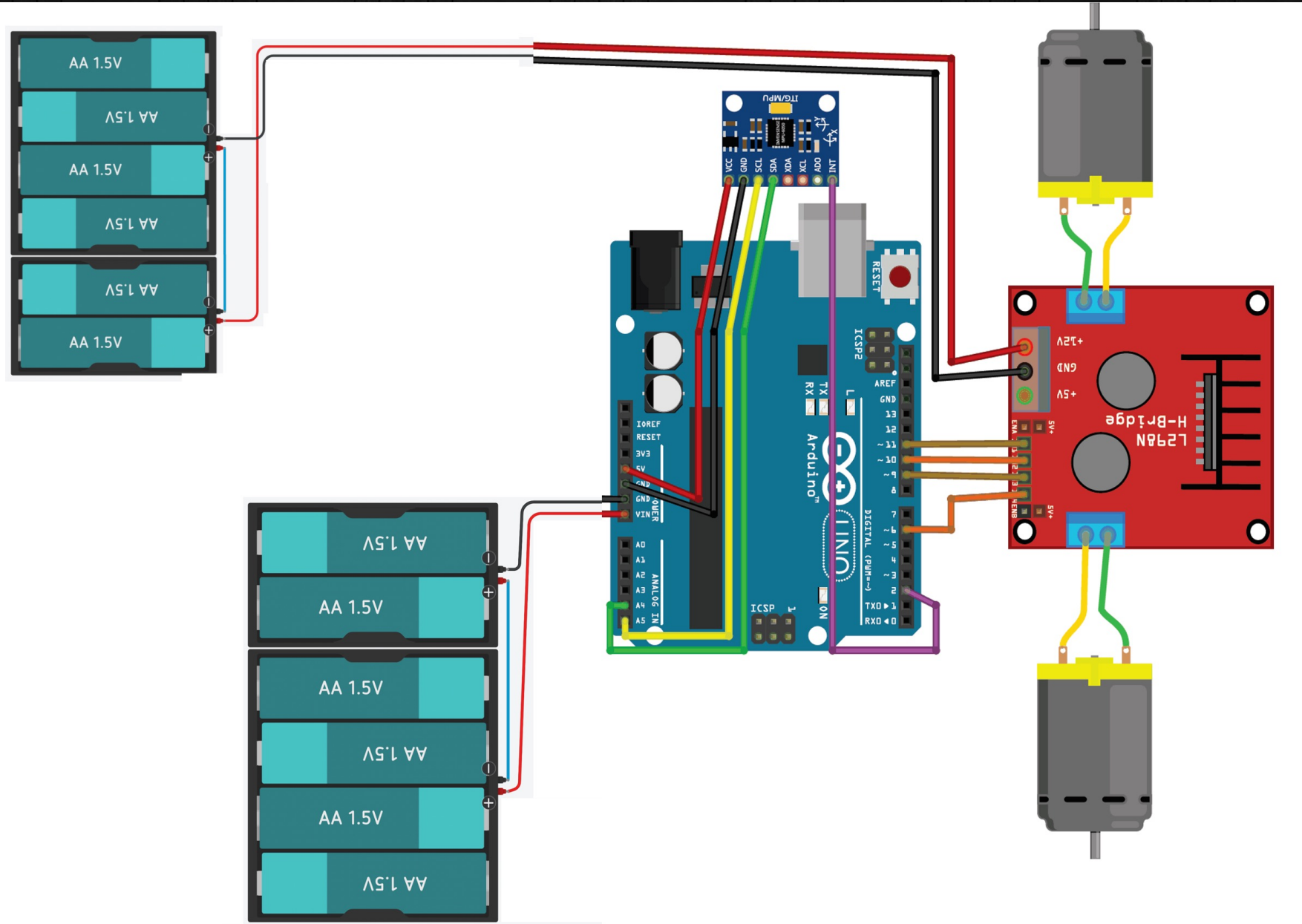
The dimensions of the Segway robot are as follows:

- i) Length = 22 cm*
- ii) Width = 15 cm*
- iii) Height = 7 cm*



Our robot meets the stipulated design criterion of width being less than or equal to 6.5 inches or 16.5 centimeters.

Electronic Circuit Diagram



Explanation of Electronic Circuit Diagram

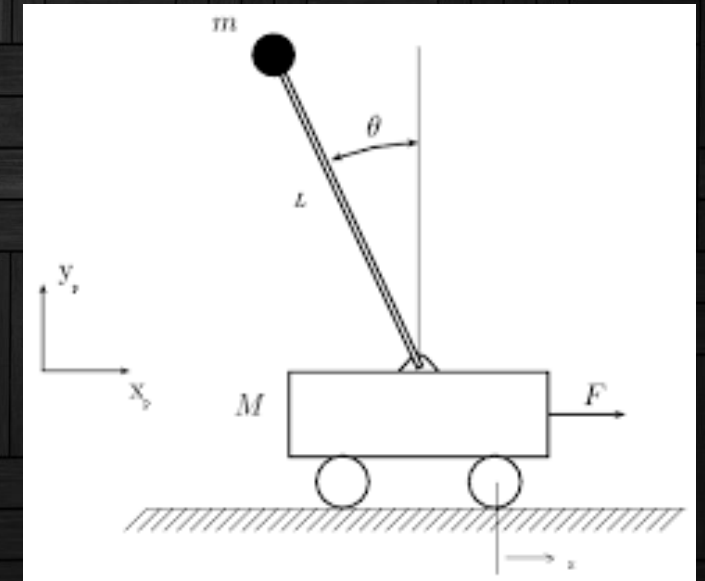
- 1) *The Arduino Uno board is powered by 6 x 1.5 V AA batteries connected in series to produce 9V and another set of 6 x 1.5 V AA batteries are connected in series to produce 9V that power the L298N motor driver. Both battery sets are connected to Vin at the positive terminal and GND at the negative terminal of both the Arduino Uno and L298N motor driver.*
- 2) *The MPU6050 sensor is powered by the Arduino Uno and is connected at 5v and GND of Arduino Uno to VCC and GND of MPU6050 respectively.*
- 3) *2 N20 12V geared motors are connected to the driver module.*

- 4) *The left motor is connected to PWM pins 6 and 9 of Arduino via M3 and M4 of driver module whereas the right motor is connected to PWM pins 10 and 11 of Arduino via M1 and M2 of driver module.*
- 5) *The SCL(Serial Clock Line) and SDA(Serial Data line) pins of MPU6050 are connected to analog input pins A4 and A5 of Arduino and INT(Interrupt) pin is connected to digital non-PWM pin 2 of Arduino.*

Mechanical Design of System

Design Principle:

The Inverted pendulum is a pendulum where the center of mass is above the pivot point. It is inherently unstable and to balance it, the robot has to move forward and backward to make the pivot point come directly under the center of mass and thus balance itself and any amount of reasonable mass that is placed on top of it.



Design Parameters and Justification:

- 1) The robot is longer compared to its width and height. This is done in order to place the components such that the setpoint or balance is as close to 180 degrees as possible without worrying about space constraints.*
- 2) Instead of using $2 \times 9V$ batteries we used $2 \times 6 \times 1.5 V$ battery packs to increase the inertia of our robot thus making it less likely to be swayed by external disturbances and thus allowing for wider margins of error while tuning, but this makes recovery beyond approximately ± 10 degrees difficult.*
- 3) There are 2 levels to this robot a lower one and an upper one held in place at 4 corners by 20 pillar screws (i.e.) 5 pillar screws for each pillar at each corner.*

- 4) *The lower level is comprised of the Arduino Uno board placed far forward, balanced by the L298n motor driver module at the top and 1 battery pack below the lower level placed just behind the axis of the N20 motors to create a balance point at approximately 183 degrees.*
- 5) *2 x 75 mm rubber tyres are connected to the DC motors by friction fit and the motors are held in place by 2 motor mounting brackets braced to the lower level.*
- 6) *The upper level consists of the other battery pack placed just behind the axis of the motors, MPU6050 at the very front for sensing tilt and the ring where the ball is balanced is placed in the space in-between , in front of the axis of the motors*
- 7) *All the components are connected by Dupont connector wires.*

- 8) *All the components in the 2 levels are placed in such a way that clockwise torque created on one side is approximately balanced by the counter-clockwise torque created by the other side. The final setpoint is about 185 degrees.*
- 9) *There are only 2 levels and not any higher to fully take advantage of the principle of inverted pendulum because increase in height can cause the robot to be easily swayed by small disturbances as it is more unstable than our design thus making tuning more difficult and less tolerant to errors. For designing such a robot with higher levels we need to place extra mass along the axis perpendicular to the axis of the motors to increase inertia, thus adding complexity and unnecessary weight.*

Controller Design

The controller used by the Arduino program to balance the robot is a Proportional-Integral-Derivative (PID) Controller in the PID_v1.h library.

i) The Proportional part of the controller is directly proportional to the error produced in the present time.

ii) The Integral part of the controller is directly proportional to the errors produced in the past and integrates them.

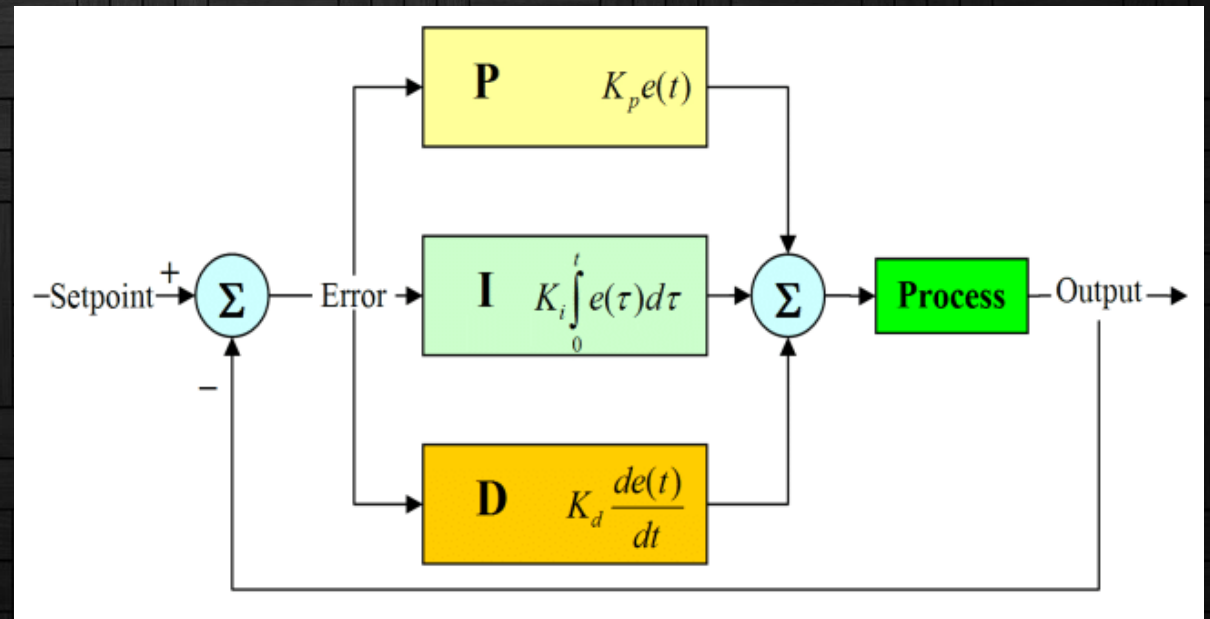
iii) The Derivative part of the controller estimates errors that can be made in the future, thus all the three controls combined provide optimal error correction to balance the Segway robot.

PID Controller

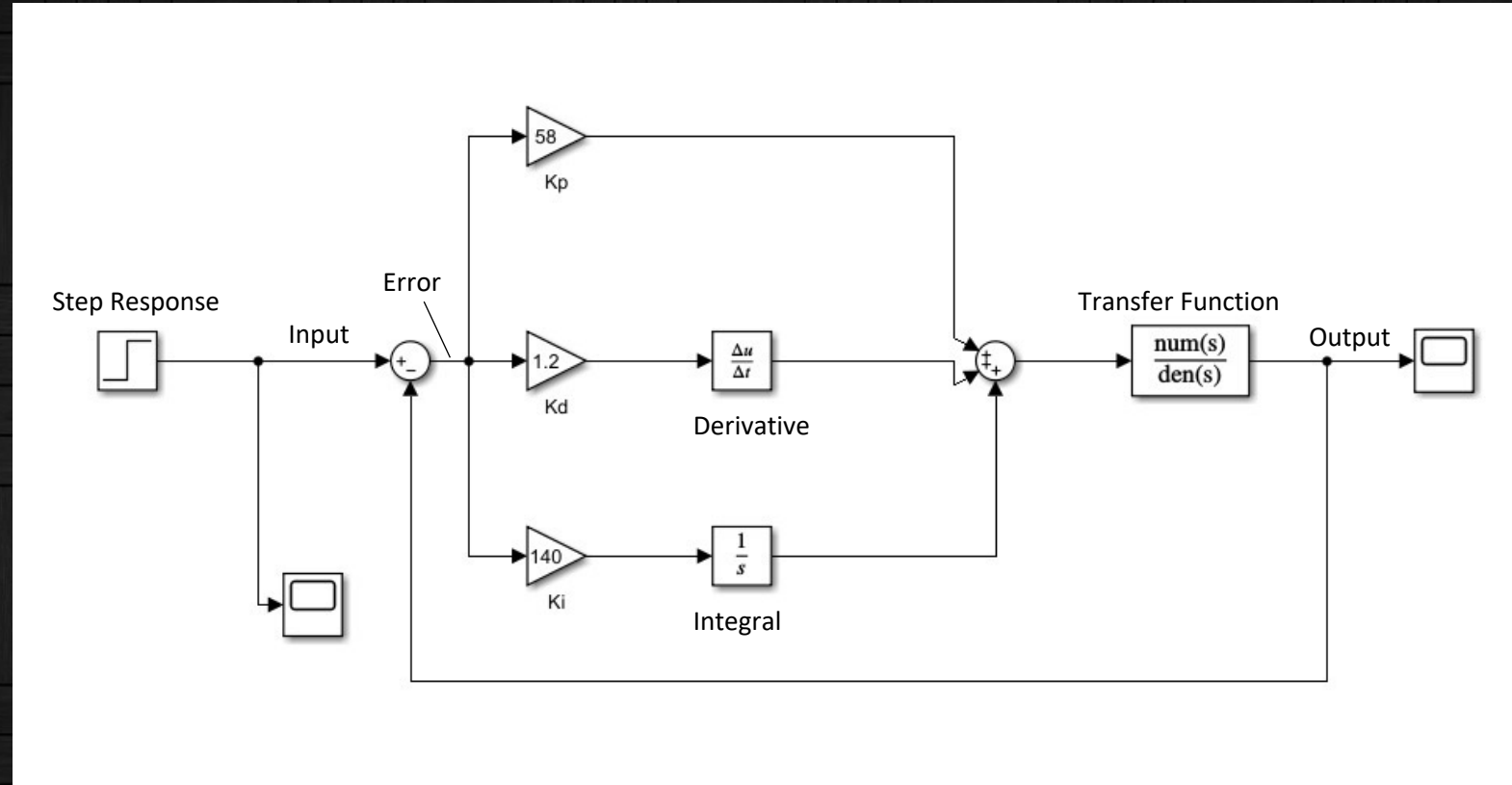
The mathematical form of the overall control function used in the PID Controller is

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t)$$

Block Diagram of PID
Controller



MATLAB Simulink Model for PID Controller



This is the Simulink Model for PID Controller, with a step response analogous to an external disturbance being given as input for the simulation to run.

Transfer Function :

The below formula is used to calculate the transfer function:

$$T(s) = \frac{X(s)}{R(s)} = \frac{K_d s^2 + K_p s + K_i}{s^3 + (10 + K_d)s^2 + (20 + K_p)s + K_i}$$

where $K_p = 58$, $K_d = 1.2$ and $K_i = 140$

$$\text{Transfer Function } \mathcal{T}(s) = (1.2s^2 + 58s + 140) / (s^3 + 11.2s^2 + 78s + 140)$$

Input:



This is the input step response.

Output:



This is the output response after passing through the simulated PID Controller.

Controller Tuning

The PID controller is tuned manually because there are so many environment variables like friction in tyres, weight shifting, slightly offset setpoint, surface, etc., that it is efficient to tune it manually with the help of MATLAB Simulink Simulation to find the approximate ballpark of where the exact values of K_p , K_i and K_d should be.

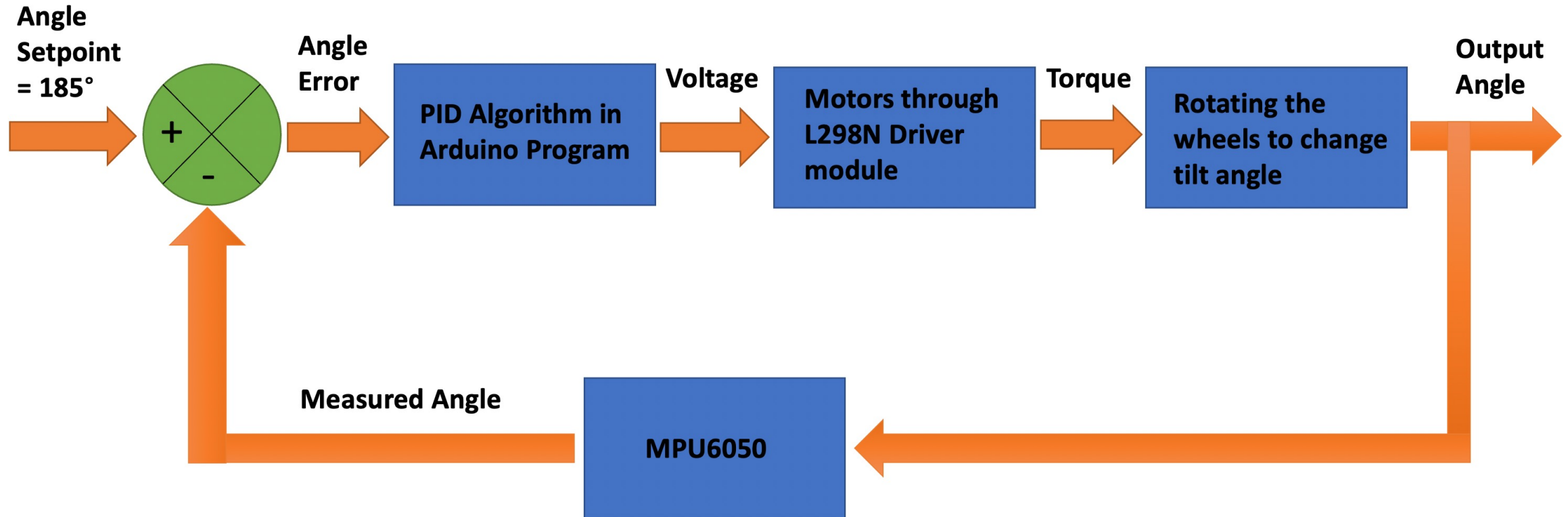
The controller is tuned in the order starting from K_p then K_d and finally K_i starting from 0 till the robot is stable.

i) *Tuning K_p* – Tuning K_p is the most difficult one. Low K_p value can result in the robot being not able to recover and High K_p value can result in the robot correcting errors far too frequently thus violently moving forwards and backwards causing unnecessary stability issues for proper balancing. $K_p = 58$ for our Segway robot.

ii) *Tuning K_d* – Tuning of K_d is comparatively easier. Low K_d value can result in very late response to tilt thus making the robot being not able to recover and High K_d value can result in over reaction to small errors thus causing stability issues for proper balancing. $K_d = 1.2$ for our Segway robot.

iii) *Tuning K_i* – K_i tuning can improve the smoothness with which the robot moves to balance itself and any object placed on top of it. $K_i = 140$ for our Segway robot.

Control System Block Diagram



We have implemented a closed loop feedback control system with the following attributes:

- i) Reference Angle is set to 185° .*
- ii) Controller – PID Algorithm in Arduino Uno producing a voltage signal*
- iii) Correction – Motors connected through L298N Driver module produce a specific amount of torque based on the voltage input from Arduino to spin the tyres.*
- iv) Process – Based on the torque from the motors, the wheels rotate in such a way that the pivot is below the center of mass of the robot for proper balancing.*
- v) Measurement element – The MPU6050 sensor measures the output angle and gives the data to the comparator. The Comparator gives the error in angle as the output to the controller.*

Arduino Program

Step- 1 : Getting the required libraries

```
sketch_may14a
#include "I2Cdev.h"
#include <PID_v1.h>
#include "MPU6050_6Axis_MotionApps20.h"
```

We get the required libraries from Git Hub that can help us communicate with the MPU6050 and PID Controller program.

[Link for MPU6050 and I2Cdev.h libraries](#)

[Link for PID_v1.h library](#)

Step- 2: Setting the setpoint and PID Controller coefficients

The setpoint is determined by gradually changing the angle of inclination of the robot until a stable position of balance is achieved. The setpoint angle is determined by reading from the MPU6050 through the serial monitor

```
Quaternion q;  
VectorFloat gravity;  
float ypr[3];  
  
double setpoint = 185;  
double Kp = 58;  
double Kd = 1.2;  
double Ki = 140;  
  
double input;  
double output;  
  
PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);
```

The values of the PID Controller coefficients K_p , K_i , K_d (found from controller tuning) are given as input and the PID program is initialized with the PID pid command.

Step- 3 : Getting the Gyroscope and Accelerometer Offsets

The offsets for the MPU6050 are calculated using the Arduino program given below and entered.

(Sensor is calibrated using code from this [website](#).)

The motor driver pins are initialized and since pins 6, 9 , 10 and 11 are PWM pins they are programmed to analogWrite and initially they are OFF (i.e.) set to LOW.

```
mpu.setXGyroOffset(-11);  
mpu.setYGyroOffset(-36);  
mpu.setZGyroOffset(11);  
mpu.setZAccelOffset(1237);  
  
pinMode(6, OUTPUT);  
pinMode(9, OUTPUT);  
pinMode(10, OUTPUT);  
pinMode(11, OUTPUT);  
  
analogWrite(6, LOW);  
analogWrite(9, LOW);  
analogWrite(10, LOW);  
analogWrite(11, LOW);  
}
```

Step- 4 : Defining functions and Producing output

The output is calculated using the PID library program and is given as instruction to rotate the tyres forwards, backwards or to stop depending upon the difference between the tilt angle and the setpoint angle. The user-defined functions Forward, Reverse and Stop are used to make the code cleaner

```
void loop() {  
    // put your main code here, to run repeatedly:  
  
    if(!dmpReady) return;  
  
    while(!mpuInterrupt && fifoCount < packetSize){  
        pid.Compute();  
        Serial.print(input);  
        Serial.print(" => ");  
        Serial.println(output);  
  
        if(input > 150 && input<200){  
            if (output > 0)  
                Forward();  
            else if (output < 0)  
                Reverse();  
            else  
                Stop();  
        }  
    }  
}
```

```
void Forward(){  
    analogWrite(6, output);  
    analogWrite(9, 0);  
    analogWrite(10, output);  
    analogWrite(11, 0);  
    Serial.print("F");  
}  
void Reverse(){  
    analogWrite(6, 0);  
    analogWrite(9, output*-1);  
    analogWrite(10, 0);  
    analogWrite(11, output*-1);  
    Serial.print("R");  
}  
void Stop(){  
    analogWrite(6, 0);  
    analogWrite(9, 0);  
    analogWrite(10, 0);  
    analogWrite(11, 0);  
    Serial.print("S");  
}
```

Code is adapted from this [website](#) . Thanks
B. Ashwinth Raj

Conclusion

This project encompasses a number of important disciplines such as IoT, Arduino Programming, Learning about PID Controllers, MATLAB Simulink, etc. When these different disciplines of ideas interact and concatenate as in the case of our “Mini Segway” robot, a plethora of learning and interactions occur thus strengthening our understanding of various concepts. The robot can balance itself and any object of reasonable weight well without much response to mild external disturbances on 2 wheels.

Team Members' Contribution

- i) *Nishant Sahoo - me20b122 => Contributed to the project by developing the MATLAB Simulink PID Control system and tuning PID Coefficients for the Segway robot.*
- ii) *M Keerthy Vasan – me20b095 => Contributed to the project by constructing the robot, Arduino programming, and making this final report.*
- iii) *Kalla Suresh – me20b093 => Contributed to the project by calculating position of components' placements and tuning PID Coefficients.*

References and Software Used:

- 1) *Course textbook*
- 2) *MATLAB Simulink and the corresponding documentation*
- 3) *Arduino IDE and the corresponding Documentation*
- 4) *Wikipedia for PID Controllers*
- 5) *[Circuit Digest website](#) for directions*
- 6) *Git Hub for implementing Arduino libraries*
- 7) *TinkerCAD for constructing the electronic circuit diagram.*
- 8) *[Google Drive](#)*



Thank you